实验记录：Text-to-SQL Agent（路线 A）

姓名：_____　　日期：2026-10-21　　环境：Windows 11 / Python 3.12.10 / openai 3.35.1

一、任务理解

构建一个能回答「学校数据库」问题的 AI Agent：用户用自然语言提问 → Agent 自主探索 SQLite 库的表结构 → 自己生成并执行 SQL → 拿到结果后再决策 → 最后用中文回答。

本任务的重点不在「写一个聊天框」，而在于跑通完整的 Agent Loop（感知—决策—调工具—回传—再决策），并让模型通过工具去「看」数据库，而不是凭训练记忆猜答案。

二、系统设计

整体由三层组成，职责单一、边界清晰：

层	文件	职责
工具层	tools.py	把「查库」封装成 3 个函数，并提供符合 OpenAI Tool Calling 规范的 JSON Schema
循环层	agent_loop.py	维护 messages 列表，驱动 模型决策 → 执行工具 → 回填结果 的循环
测试层	test_tools.py / test_queries.py	先离线验证工具，再跑题目指定的 5 个问题并存档轨迹

一次问答的实际数据流：

```
用户：张老师带的班里，编程成绩前三名是谁？
  │
  ├─[第 1 轮] 发出 system prompt + 问题 + 3 个工具的 JSON Schema
  │        模型返回 tool_calls = [list_tables]          ← 模型自己决定的
  │        我们执行 → {"tables":[...], "row_counts":{...}}
  │
  ├─[第 2 轮] 追加 {role:"tool", tool_call_id, content:结果}
  │        模型返回 tool_calls = [get_table_schema × 4]
  │        分别查 teachers / classes / students / scores
  │
  ├─[第 3 轮] 模型先验证「张老师带哪些班」→ 三班、四班
  │
  ├─[第 4 轮] 模型执行 JOIN 查询 → 韩八 94 / 朱十 92 / 吕四 89
  │
  └─[第 5 轮] 模型不再用工具，出中文回答
```

说明：模型本身不保存状态，每一轮都要把完整的历史重新发一遍；工具结果必须以 role="tool" + tool_call_id 回填，模型才能把结果与请求对上号。

三、关键实现

3.1 工具设计

三个工具刻意做成「由粗到细」的探索顺序，避免模型一上来就猜列名：

工具	作用	设计考虑
list_tables	列出业务表及各表行数	过滤 sqlite_ 开头的内部表（sqlite_sequence 会干扰模型判断）
get_table_schema	列定义 + 外键 + 2 行样例	样例数据让模型知道 subject 存的是「数学」而不是「math」
execute_sql_query	执行只读 SQL	返回 {columns, rows}，出错时返回 SQLite 原文

3.2 三道安全锁

「只让模型查、不让它改」不能只靠一句 Prompt 约束，代码里做了三层防御：

层次	手段	拦截效果
① 语句校验	首关键字必须是 SELECT / WITH，且不允许分号拼接多条	拦住 DROP / DELETE / UPDATE / INSERT
② 只读连接	sqlite3.connect(uri="file:...?mode=ro")	即使校验被绕过，SQLite 层面拒绝写入
③ query_only	PRAGMA query_only = ON	连接级的双保险

实测拦截了 6 类危险语句（含 SELECT 1; DROP TABLE students 这种拼接攻击），攻击后数据库仍为 30 人，数据完好。

3.3 Agent Loop

```
messages = [system, user]
for step in range(1, MAX_STEPS + 1):
    resp = client.chat.completions.create(model, messages, tools=TOOL_SCHEMAS,
tool_choice="auto")
    msg  = resp.choices[0].message
    messages.append(assistant_message(msg))

    if not msg.tool_calls:            # 模型不要工具了 → 已有答案
        return msg.content

    for call in msg.tool_calls:       # 可能一次要调多个，逐个执行逐个回填
        result = dispatch(call.function.name, call.function.arguments)
        messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
```

3.4 Self-Correction（自我修正）

关键设计：工具执行失败时不抛异常，而是把 SQLite 的原始报错字符串回传给模型。抛异常会直接打断循环，模型永远看不到错在哪；返回原文，模型下一轮就能读到 「no such column: namee」并自己改对。所谓 Self-Correction，本质是我们给它开了一条「能读到错误」的反馈通道。

3.5 防死循环

MAX_STEPS = 8。取 8 的依据：第 3 题需要「列表 → 看 4 张表结构 → 2 次验证查询 → 1 次主查询」共约 8 次工具调用，最少要留 4~5 轮给探索和纠错。步数耗尽后摘掉 tools 参数再请求一次，强制模型基于已有信息收尾，而不是直接抛错。

四、测试与结果

4.1 工具层单元测试（离线，不消耗额度）

共 22 项断言，全部通过：表清单正确、sqlite_sequence 被过滤、外键能读出、行数统计正确、6 类危险语句全部拦截、攻击后数据完好、错误信息可回传。

```
共 22 项，通过 22 项，失败 0 项
```

4.2 五个测试问题

#	问题	Agent 回答（摘要）	轮次	工具调用次数
①	全校有多少学生？	30 名学生	4	3
②	数学平均分是多少？	75.67 分（30 条成绩）	4	3
③	张老师班的编程前三？	韩八 94 / 朱十 92 / 吕四 89	5	8
④	哪个班平均成绩最高？	一班 78.26（并主动说明口径）	4	5
⑤	有没有缺考的学生？	王明缺编程、卫五缺历史	5	6

5 题全部答对，平均耗时 5.9 秒，最快 3.8 秒、最慢 8.1 秒。

4.3 三道「坑」的实测表现

出题人埋了三处陷阱，实测中 Agent 的处理情况：

陷阱	Agent 的表现
张老师带两个班（三班+四班）	先查 teachers 再查 classes，确认「张老师 = id 3」，发现对应三班、四班，因此第三名取到四班的吕四 89 分（若只看一个班会错成 85 分）
缺考 = 成绩记录少于 5 门	先用 COUNT 按学生统计记录数，再构造「全体学生 × 全部科目」的组合做 LEFT JOIN，精确得到 2 条缺失：王明-编程、卫五-历史
「班平均分」的口径分歧	算出「按成绩条平均」的结果后，主动指出一班 78.26 与三班 78.20 只差 0.06，并说明各班记录数不同（34/30/20）会让口径影响排名，提示可换成「按学生平均再平均」对比

五、遇到的问题与解决

问题 1：程序一直报「配置缺失」，但 .env 里明明填了 Key

现象：verify_api.py 报「没找到真实 Key」，而文件里肉眼能看到 sk- 开头的字符串。排查顺序：先怀疑编码（记事本可能存成 UTF-16/BOM）→ 检查前 12 字节，确认是 UTF-8 无

BOM，排除；再用 python-dotenv 单独解析，发现解析出的 DEEPSEEK_API_KEY 长度是 17，正好等于占位符 your_api_key_here 的长度。逐行打印（值做掩码）后真相大白：Key 被粘到了第 1 行的**注释**里，而真正生效的赋值语句仍然是占位符。

解决：从整份文件中提取 sk- 开头的字符串，重建一份只含三行有效赋值的 .env，重新验证通过。教训：注释行里的「赋值」永远不会生效，改配置要看的是生效行，不是看得见的那行。

问题 2：Windows 下中文输出乱码

cmd 默认代码页是 936(GBK)，Python 打印中文时容易变成乱码，把日志重定向到文件时更明显。解决：在脚本入口处 sys.stdout.reconfigure(encoding="utf-8")（与老师提供的 init_db.py 做法一致），保证终端输出和重定向日志都是 UTF-8。

问题 3：PowerShell 执行策略为 Restricted，激活虚拟环境被拒

报错「因为在此系统上禁止运行脚本」。解决：VSCode 里改用 Command Prompt 作为默认终端，或执行 Set-ExecutionPolicy -Scope CurrentUser RemoteSigned。这条也在避坑清单里提醒了自己。

问题 4：openai 包版本较新（3.35.1），担心 API 不兼容

没有凭印象判断，而是直接检查：打印 openai.__version__ 与模块路径，并用 inspect.signature 确认 client.chat.completions.create 仍然支持 tools / tool_choice 参数（41 个参数中均存在）。结论：写法无需调整。

六、不足与改进方向

不足	改进思路
第 3 题一次连调 4 个 get_table_schema，浪费往返	把「一次查全部表结构」做成一个批量工具，或对 schema 做会话内缓存
每道题都从 list_tables 重新探索	在同一会话内复用已探索的 schema，减少 token
「平均分」等模糊口径只能事后说明	在执行前增加一次「澄清追问」，让 Agent 主动问用户要哪种口径
无流式输出，等待期无反馈	openai SDK 支持 stream=True，可边生成边打印
报错重试没有次数上限	对同一工具的连续失败设置次数上限，避免模型绕圈

七、附：完整运行日志

每次运行的完整轨迹（每一轮的思考、动作、观察）自动存档在项目 logs/ 目录下，文件名为 run_<时间戳>.md 与 .json。本次运行的日志文件：

```
logs/run_20261021_194314.md
logs/run_20261021_194314.json
```

仓库地址与提交记录见 Git 提交历史（首次提交含全部源码与本次运行日志）。